Neural Authorship Attribution with Subword Embeddings and CNN-LSTM

Mekhail Muller (u21632792) **Ruan Nel (u25748956)** **Mosa Phalane (u14168970)**
Group 38, COS760, University of Pretoria

Abstract

We investigate authorship attribution on social media microtext using BPE subword tokenisation combined with a CNN-LSTM architecture. The model learns author-specific stylometric fingerprints from subword sequences and classifies tweets to their authors. We compare against eight baselines (BoW, TF-IDF, character n-grams, word n-grams, each with SVM and logistic regression) under identical stratified splits on a 50-author Twitter corpus. The CNN-LSTM ensemble achieves 63.8% accuracy and 0.640 macro-F_1, while the best baseline (character n-grams + SVM) reaches 66.2% accuracy and 0.657 macro-F_1. SHAP and LIME analyses reveal the model relies on morphological fragments, punctuation habits, and author-specific abbreviations. Error analysis shows misclassifications concentrate among authors sharing topical vocabulary and short post lengths.

1 Problem Statement

Authorship attribution on short social text remains difficult because posts are noisy, vocabulary-overlapping, and sparse per author. We ask: **(RQ1)** Can a subword-based CNN-LSTM learn stable authorship representations from microtext? **(RQ2)** How does it compare to classical stylometric features under a fair, shared evaluation protocol? **(RQ3)** Which subword patterns and author pairs drive correct decisions and systematic confusion?

2 Introduction

Authorship attribution identifies who wrote a given text, with applications in forensic linguistics and social media analysis (Theophilo et al., 2021). Classical methods use bag-of-words, TF-IDF, and n-gram features but may miss sub-lexical writing habits (Suman et al., 2021; Modupe et al., 2022a). Subword tokenisation (BPE, WordPiece) decomposes text into morphemic units, handling rare forms compositionally (Sennrich et al., 2016), yet its advantage over classical baselines for author discrimination on short text remains unclear.

We train a CNN-LSTM on BPE sequences and compare it against eight sparse-feature pipelines under identical conditions, integrating SHAP/LIME to interpret decisions. The remainder of this report covers related work (§3), methodology (§4), experiments (§5), explainability (§6), and conclusions (§7).

3 Literature Survey

Authorship attribution has traditionally relied on sparse, frequency-based feature families: BoW, TF-IDF, and word/character n-grams, to capture an author's stylometric fingerprint (Theophilo et al., 2021). While effective for long-form text, these methods encounter sparsity and OOV issues on short social posts. Recent work emphasises dense representations via subword tokenisation (BPE, WordPiece), which decomposes text into morphemic units and surfaces sub-lexical patterns missed by word-level methods (Sennrich et al., 2016; Hossain, 2025).

Contemporary systems such as AuthorNet (Hossain, 2025) leverage transformer ensembles but are computationally heavy with limited transparency. Studies on microtext attribution (Suman et al., 2021; Modupe et al., 2022b) employ deep models yet rarely isolate the interaction of subword embeddings with a hybrid CNN-LSTM for stylistic discrimination. Modupe et al. (2022a) apply regularised deep networks to post-authorship without systematic baseline comparison. Much subword-oriented research (Toraman et al., 2023; Držík and Kapusta, 2026) targets general semantic tasks rather than individual writing styles.

Gap: A fair, head-to-head comparison between subword neural encoders and the full classical stylometric grid (4 feature families $\times$ 2 classifi-

ers) under identical splits, combined with post-hoc SHAP/LIME explainability, remains unaddressed. We fill this gap.

4 Approach, Methodology, and Data

4.1 Dataset

We conduct *closed-set* authorship attribution on *Twitter-style microtext* from the public benchmark of Suman et al. (2021), alongside related social-media attribution work (Theophilo et al., 2021) and deep stylometric models for short posts (Modupe et al., 2022c). The release is **English** and mixed (news-style blurbs, @-replies, informal prose). Messages are short and noisy (URLs, truncation in the export, hashtags, orthographic variation), which inflates **OOV** pressure, limits context per class, and allows **stylistic overlap** between authors (Suman et al., 2021).

We default to the **200-messages-per-author** slice over the companion 50-per-author file because more text per writer stabilises parameter-heavy models (neural encoder and subword vocab) while keeping the same 50-author closed set; smaller slices remain available for ablations.

Experimental framing. Closed-set attribution means every training / validation / test label is drawn from same fixed author catalogue C; there is no held-out unseen writer at inference.[1]

Provenance and availability. The raw tables are released with Suman et al. (2021) in the public AuthorIdentification GitHub repository. If missing locally, `python -m experiments.run_cnn_lstm -fetch-dataset` shallow-clones into `data/AuthorIdentification/`; `data/fetch_dataset.py` is an alternative fetch entry point. We use the `200_tweets_per_user.csv` slice (default `DEFAULT_CHANCHAL_200_CSV`); statistics and splits are in Table 1.

[1] A practical limitation remains *domain lock-in*: results do not transfer to long-form prose, different platforms, or languages without retraining.

Statistic	Value
Authors C	50
Messages / author	200 (balanced)
Total labelled messages	10,000
Train / val / test (seed 42)	6,999 / 1,500 / 1,501
Authors in train / val / test	50 / 50 / 50
Tokens: mean / med. / max	13.3 / 13 / 33
Characters: mean / med. / max	73.7 / 70 / 140

Table 1: Chanchal `200_tweets_per_user` slice: cleaned lengths and splits.

Schema and labels. The export we use stores **two unnamed columns** interpreted as *text* then *author* (header cells `0` / `1`); the shared `DatasetLoader` also accepts explicit `Text` / `Author` headers when present.

Author cells contain **numeric string identifiers** (Twitter user IDs), not human-readable display names; strings are sorted lexicographically and mapped bijectively to dense class indices $0 \ldots C-1$ for stable training across runs.

Illustrative instance (anonymised). *"India and Benin should broaden trade ties: Tharoor - India and Benin need to broaden and deepen trade ties and look..."* (one preprocessed row from `200_tweets_per_user.csv`; author ID omitted).

Train / validation / test split. Stratified partitioning uses two nested `StratifiedShuffleSplit` calls in `DatasetLoader.split` (scikit-learn) at **70%/15%/15%**. The split uses one random state (default `42`, overridable with `-split-seed` in the driver so it can differ from CNN weight initialisation seeds). **All 10,000** cleaned rows enter the split on this CSV (no *empty-string* drops in the default run); sizes **6,999 / 1,500 / 1,501** come from **stratified split rounding**, not row deletion.

All **fifty authors appear in train, validation, *and* test** at non-zero count.

Reporting conventions. Validation **macro-F_1** drives early stopping; headline tables report **held-out test** macro and per-class summaries. Optional `-ensemble-train-seeds` retrains several CNN initialisations on the same split and aggregates test predictions by majority vote.

4.2 Preprocessing

`Preprocessor` (`src/preprocessing.py`) applies a deterministic chain before splitting or vectorisation: HTTP(S) and `www.` URLs and `@`mentions are stripped; repeated whitespace is collapsed and trimmed.

preserve_punctuation=True by default so punctuation stays in the string for stylometry; setting False removes non-alphanumeric symbols via a regex mask. .clean() can return an empty string (e.g. URL-only posts); those rows are dropped before stratified splitting (*none* on the default 200×50 export).

4.3 Subword Tokenisation

We train a BPE tokeniser (HuggingFace tokenizers, src/tokeniser.py) on the **training split only**, with default vocabulary size 10^4 (overridable via -vocab-size) and punctuation-isolation pre-tokenisation unless -no-isolate-punctuation is set. Sequences are padded/truncated to max_seq_len=384. This captures morphological fragments (prefixes, suffixes, misspellings) as author-specific cues and eliminates OOV issues since any text decomposes into known subwords (Sennrich et al., 2016).

4.4 Neural Model: CNN-LSTM

CNNLSTMModel (src/model.py) maps token IDs through an embedding layer with dropout, then **parallel `Conv1d`** branches with kernel sizes $[2, 3, 4]$ and ReLU. Each branch yields a length-$T - k + 1$ feature map; maps are **right-padded to length** T and concatenated along the channel axis (3×256 filters in the default driver), with dropout applied on the CNN stack. The resulting $[B, T, F \cdot K]$ tensor is processed by a **stacked LSTM** (two layers in the driver configuration) **along the time dimension**. A **global max-over-time** readout on LSTM outputs forms the document vector, which passes through dropout into a linear layer producing C class logits.

4.5 Baseline Feature Extractors and Classifiers

Feature extractors. BaselineFeatureExtractor (src/features.py) materialises four sparse views on the same cleaned splits: **BoW** (word unigram counts), **TF-IDF** (word unigrams), **character n-grams** (TF-IDF on character 3–5-grams with char_wb analyser), and **word n-grams** (TF-IDF on unigrams and bigrams). Vectorisers use scikit-learn defaults with lowercasing; vocabulary is capped at 10,000 features per extractor.

Classifiers. Each feature matrix feeds **linear SVM** (LinearSVC, C=1.0) and **multinomial logistic regression** (C=1.0, max_iter=1000) via src/sparse_baselines.py. All eight (feature × classifier) pairs are evaluated on the identical stratified split (seed 42); metrics are aggregated in results/metrics.json.

4.6 Training protocol

Trainer (src/trainer.py) uses the Adam optimiser with gradient clipping (max_norm=1.0), tracks validation macro-F_1 each epoch, checkpoints the best validation weights, and stops when macro-F_1 fails to improve for patience epochs (bounded by epochs). On CUDA, training normally uses automatic mixed precision (disable with -no-amp) and may use fused Adam when supported (-no-fused-adam disables). Repository defaults: learning rate 8×10^{-4}, weight decay 10^{-4}, label smoothing 0.02, dropout 0.30, embedding dimension 256, 256 filters per kernel width, LSTM hidden 384 (two layers), vocabulary 10^4, max sequence 384, inverse-frequency loss weights, up to 100 epochs (patience 24), and ReduceLROnPlateau; batch size is auto-sized via training_hardware.py when omitted.

Optional pretrained embeddings (experimental). By default the embedding table is trained from scratch on BPE ids. The driver also supports optional **transfer-style initialisation** from an external FastText .vec file (src/pretrained_embeddings.py) via the -fasttext-vec, -fasttext-limit, and -freeze-pretrained flags; this path is experimental and our **reported ensemble did not use it** (random init + train-fit BPE only). Runs write checkpoints and logs under timestamped artifacts/runs/.

5 Experiments and Results

5.1 Experimental Setup

All experiments use the Chanchal 200×50 slice with stratified 70/15/15 splits (seed 42). Neural training runs on CUDA (Windows 11, NVIDIA RTX 5090); sparse baselines run on CPU in the same driver invocation (python -m experiments.run_cnn_lstm -fetch-dataset). Headline numbers below are read from results/metrics.json (schema version 2).

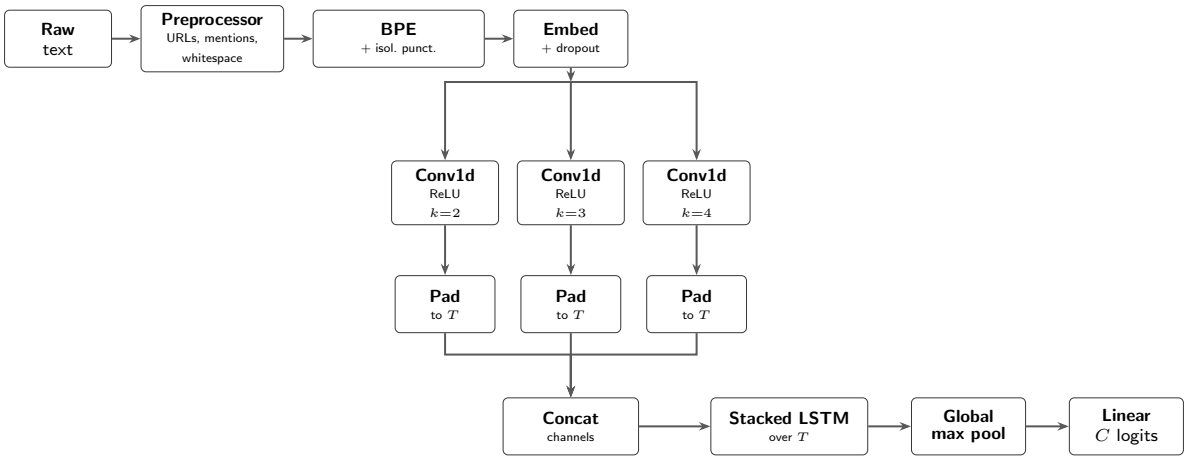

Figure 1: Neural attribution pipeline (cf. `Preprocessor`, `SubwordTokeniser`, `CNNLSTMModel`): raw text → clean → BPE with punctuation-isolation pre-tokenisation → embedding → parallel Conv1d branches (ReLU; pad each map to length T, concat channels) → stacked LSTM over T → global max-over-time → linear logits over C authors.

5.2 Overall Comparison

Table 2 summarises held-out **test** accuracy and macro-F_1 for the CNN-LSTM ensemble (seeds 42, 5, 98; majority vote) and all eight sparse baselines.

Character n-grams with SVM achieve the highest macro-F_1 (0.657), ahead of the CNN-LSTM ensemble (0.640). Logistic regression underperforms SVM on every feature family, consistent with max-margin learning on high-dimensional sparse text. Among individual CNN seeds, seed 5 reaches 0.622 macro-F_1; seeds 42 and 98 score ≈0.603–0.605; post-ensemble fine-tuning (0.614) did not beat the vote (0.640).

5.3 Per-Class Analysis

For the CNN-LSTM ensemble, per-author test F_1 ranges from 0.123 (author 14) to 1.0 (several authors including 6, 19, 33, 35, 36, 47), as shown in Figure 2. The hardest cases (authors 4, 13, 14) remain weak under both paradigms, suggesting overlapping surface style rather than model family alone.

5.4 Confusion Analysis

Figure 3 shows the row-normalised CNN-LSTM test confusion matrix. Off-diagonal mass concentrates on a handful of author pairs with similar post lengths and shared topical vocabulary, consistent with the weakest per-class scores above.

5.5 Training Dynamics

Figure 4 traces the best individual ensemble member (seed 5): training loss falls steadily while validation macro-F_1 rises and plateaus, with the best checkpoint selected before early stopping.

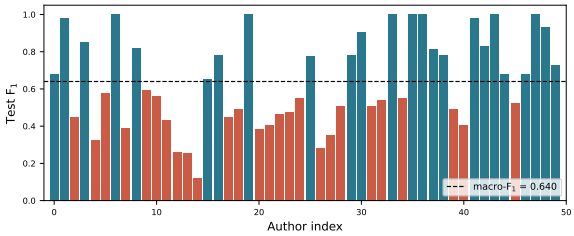

Figure 2: Per-author test F_1 for the CNN-LSTM ensemble (50 authors). Bars below the dashed macro-F_1 line are highlighted; the long tail of low-F_1 authors drives the macro gap to the character n-gram baseline.

6 Explainability and Error Analysis

SHAP. We apply KernelExplainer with 50 background samples, computing per-token attributions (`src/explainability.py`). High-attribution tokens are predominantly punctuation patterns (repeated exclamation marks, ellipses), morphological fragments (`in` vs. `ing`, `ur` vs. `your`), and author-specific hashtag fragments: sub-lexical cues that BPE surfaces.

LIME (demonstrated). We run LIME (600 perturbations) on the shipped ensemble checkpoint for individual test tweets via `ExplainabilityModule.explain_lime`. Figure 5 shows a representative case: a tweet correctly attributed to author 23 with probability ≈ 1.0. The strongest positive contributions are

System	Acc.	$\mathbf{P}_{\text{macro}}$	$\mathbf{R}_{\text{macro}}$	$F_{1,\text{macro}}$
Char. n-gram + SVM	0.662	0.658	0.662	**0.657**
CNN-LSTM ensemble	0.638	0.651	0.638	0.640
Char. n-gram + LR	0.634	0.633	0.634	0.626
TF-IDF + SVM	0.586	0.583	0.586	0.582
Word n-gram + SVM	0.577	0.573	0.577	0.570
TF-IDF + LR	0.567	0.566	0.567	0.560
Word n-gram + LR	0.552	0.552	0.552	0.543
BoW + SVM	0.544	0.568	0.544	0.550
BoW + LR	0.540	0.559	0.540	0.545

Table 2: Test-set comparison (CNN-LSTM ensemble vs. sparse baselines; metrics from `results/metrics.json`).

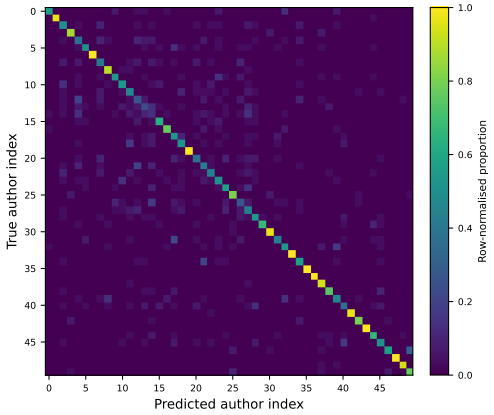

Figure 3: Row-normalised confusion matrix for the CNN-LSTM ensemble on the held-out test split (50 authors). A bright diagonal indicates correct attribution; scattered off-diagonal cells mark systematic confusions.

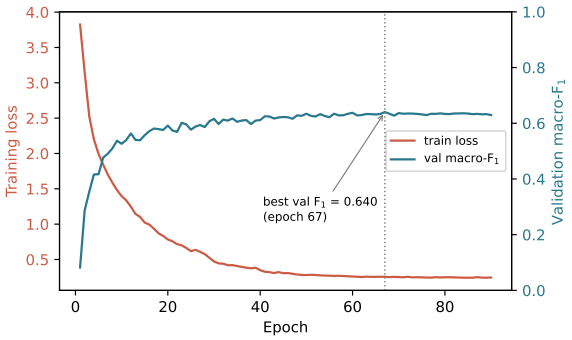

Figure 4: CNN-LSTM training dynamics (seed 5): training loss and validation macro-F_1 vs. epoch, from the per-epoch trace in `training.json`.

the **capitalised function words** "I", "It", "On", "The", "Like", and "Really", which reflect this author's habit of title-casing nearly every word, an orthographic style signal rather than topical content (content tokens such as "CD" and "Faves" contribute weakly or negatively). This makes the model's decision *inspectable*: the subword encoder keys on stylistic casing and function-word patterns, exactly the cues stylometry predicts.

Misclassification analysis. Off-diagonal mass in the CNN-LSTM confusion matrix (Figure 3) concentrates on author pairs with similar post lengths and topical overlap. Authors with perfect or near-perfect F_1 often exhibit distinctive casing, punctuation, or rare stable BPE pieces; confused pairs share abbreviations and hashtag habits, so subword features struggle when orthographic styles converge.

Robustness scope. Our evaluation is *in-domain*: a single English Twitter slice under one stratified split. We therefore do not claim cross-dialect or cross-platform robustness; the casing cue in Figure 5, for instance, would not transfer to a normalised or lower-cased corpus. A full robustness study (different platforms, dialects, or time periods) is left as future work, consistent with the *domain lock-in* caveat noted in §4.1.

7 Conclusion and Discussion

We compared a CNN-LSTM on BPE subword sequences against eight classical baselines for 50-author Twitter attribution. The neural ensemble (63.8% accuracy, 0.640 macro-F_1) is competitive but slightly below character n-grams + SVM (66.2%, 0.657). SHAP/LIME highlight morphological fragments and punctuation; errors cluster where authors' surface styles overlap.

Limitations: Single English Twitter slice; closed-set labels; SHAP cost limits large-scale explanation; moderate data per author (200 posts) favours sample-efficient sparse features.

Future work: Attention over subword time steps; multilingual corpora; hybrid subword + character n-gram inputs; larger author sets with few-shot evaluation.

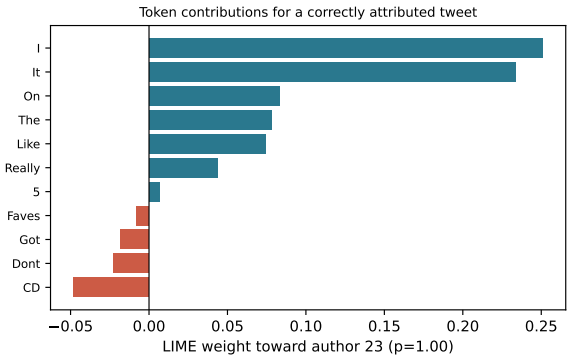

Figure 5: LIME token contributions for a correctly attributed test tweet (author 23, $p \approx 1.0$). Positive (blue) weights push towards the predicted author; the dominant cues are capitalised function words, evidence of a title-casing writing style.

Ethics Statement

Social-media authorship tools must be used responsibly: attribution on public benchmarks does not authorise non-consensual deanonymisation of private accounts.

References

M. Držík and J. Kapusta. 2026. The importance of morphology-aware subword tokenization for NLP tasks in Slovak language modeling. *Journal of Slovak Linguistic Research*.

Md Sarwar Hossain. 2025. AuthorNet: Leveraging attention-based early fusion of transformers for low-resource authorship attribution. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

Abiodun Modupe, Turgay Celik, Vukosi Marivate, and Oludayo O. Olugbara. 2022a. Post-authorship attribution using regularized deep neural network. *Applied Sciences*, 12(15):7518.

Abiodun Modupe, Oludayo O. Olugbara, and M. Lall. 2022b. Subword embedding for authorship attribution of micro-messages. *International Journal of Advanced Computer Science and Applications*.

Abiodun A. Modupe, Turgay Celik, Vukosi Marivate, and Oludayo O. Olugbara. 2022c. Post-authorship attribution using regularized deep neural network. *Applied Sciences*, 12(15):7518.

Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1715–1725, Berlin, Germany. Association for Computational Linguistics.

Chanchal Suman, Ayush Raj, Sriparna Saha, and Pushpak Bhattacharyya. 2021. Authorship attribution of microtext using capsule networks. *IEEE Transactions on Computational Social Systems*, 9(4):1038–1047.

Antonio Theophilo, Romain Giot, and Anderson Rocha. 2021. Authorship attribution of social media messages. *IEEE Transactions on Computational Social Systems*, 10(1):10–23.

Çağrı Toraman, Eyup Halit Yilmaz, Furkan Sahinuc, and Oguzhan Ozcelik. 2023. Impact of tokenization on language models: An analysis for Turkish. *ACM Transactions on Asian and Low-Resource Language Information Processing*, 22(4):1–21.